# Do we need more bikes?
# Project Report in Machine Learning

**Junjie Hu** **Chengyang Wang** **Yue Zhou** **Huan Li**

**Abstract**

In this project, we address a binary classification task to predict whether the number of bikes in a bicycle-sharing system should be increased based on various factors. Using a training dataset of 1,600 entries, we analyzed and preprocessed the data, explored five families of classification methods (Logistic Regression, Discriminant Analysis, K-Nearest Neighbors, Tree-Based Methods, and Boosting), and selected QuadraticDiscriminantAnalysis as the most suitable model for this problem. Our choice was based on an evaluation using weighted F1-scores, with the model intended for application to future data.

## 1 Introduction

Public bicycle-sharing systems provide a sustainable and convenient solution for travel. However, ensuring an actionable and accurate supply of bicycles is a vital problem in urban mobility. In Washington D.C., this challenge requires efficient management to align bike availability with demand, which is influenced by factors such as weather, time of day, and temperature. This project addresses the issue by formulating it as a binary classification problem: predicting whether bike availability should be increased at specific times.

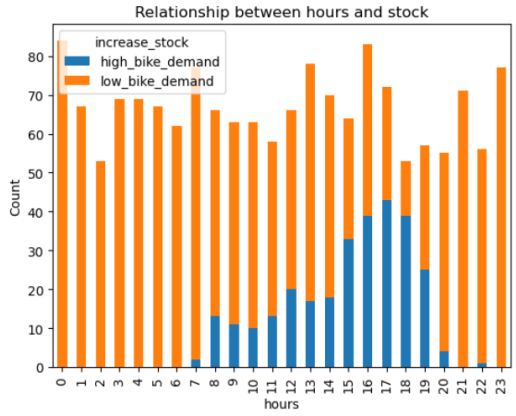
The objective of this project is to develop and evaluate classification models to predict bike availability needs accurately.To approach the problem, we start by analyzing the type and trend of the training data. The training data includes 1,600 rows, and we split it in a ratio of 0.8/0.2 to keep some of our data for validation.

## 2 Data analysis process

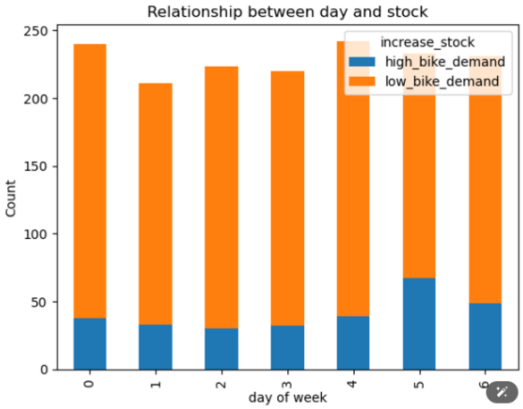
**Numerical Features:** temp, dew, humidity, precip, snow depth, windspeed, cloudcover, visbility.

**Categorical Features:** hour of day, day of week, month, holiday, weekday, summertime, snow.

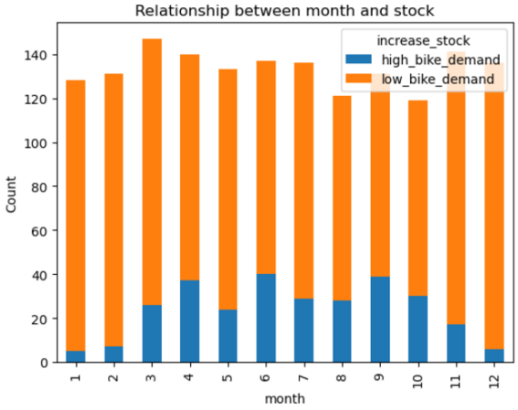
**2.1 About hour, day and month:** Looking at figure(a), it is obvious that during the day, high bicycle demand is more distributed, especially in the afternoon (hours 15-18). At night, there is no high demand for bikes at all. That's why we decide to devide 'hour of day' into day(8:00 - 19:00) and night(20:00 - 7:00). In terms of figure(b), it is clear that the demand for bicycles is higher on Saturdays and Sundays than on weekdays so we decide to delete 'day of week' but reserve 'weekday'.And in figure(c), obviously Jan, Feb and Dec(winter) have the lowest bike demand in the whole year. From March to November, the distribution is uneven. Considering that January is connected to December, we consider converting the month to a value by trigonometric function, in this way we can transfer categorical feature to numerical feature.



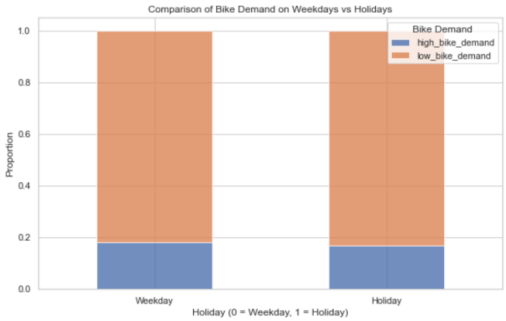

(a) Chi-Square Distribution Plot of hours



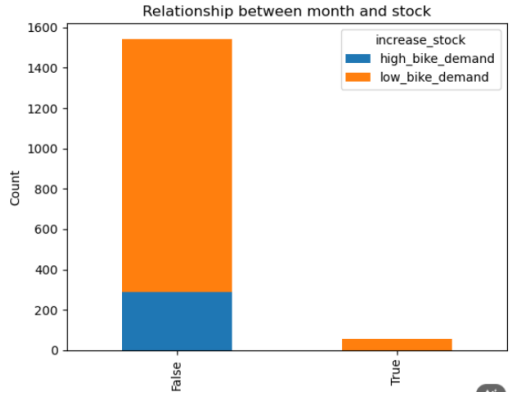

(b) Chi-Square Distribution Plot of day



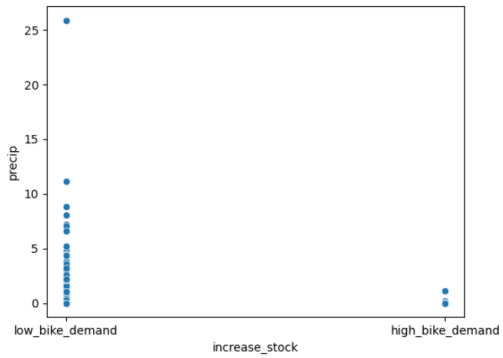

(c) Chi-Square Distribution Plot of months




(d) weekday compare holiday




(e) trend depending on snowy day




(f) trend depending on precip

Figure 1: Chi-square analysis of Categorical features

**2.2 About holiday: Looking at figure(d):** it is clear that there is no difference between weekday and holiday. So we decide to delete holiday.

**2.3 About snow: Looking at figure(e):** we use snow depth to update snow (snowdepth != 0 then snow = true), we can see that when snowy, there is no high bike demand. Although the amount of data is relatively small, we decide to reserve it.

**2.4 About precip: Looking at figure(f):** we find the high bike demand may relate to precip cince there is no high precip in group of high bike demand. However, with very low precip, there are still a lot of low bike demand. So we think it might relate to precip, but precip is not the deciding factor.

In the beginning, we decided to keep the precip, but in the model training prediction later, we find the model performed better after removing the precip.

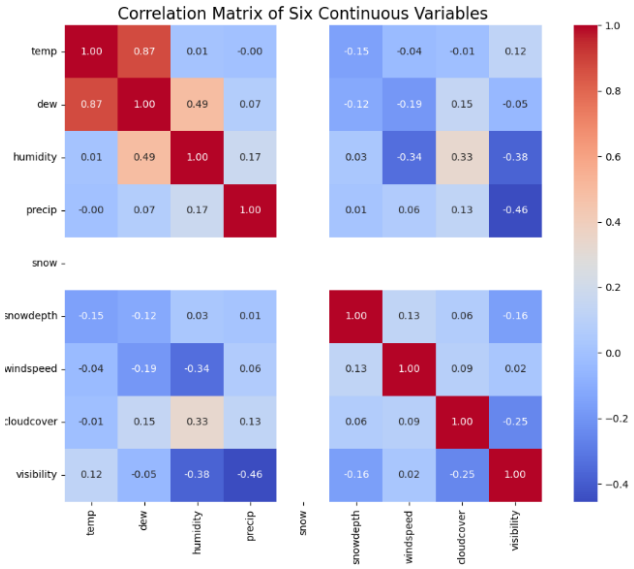



Figure 2: Correlarion matrix of Numerical features

We conducted correlation analysis for the remaining numerical features, and the results are shown in Figure 2. It is obvious that the correlation coefficient between temp and dew up to 0.87. Besides, correlation coefficient between dew and humidity up to 0.49. So we decide to delete 'dew' and reserve 'temp'. In terms of humidity, although it has some correlation with windspeed, cloudcover and visibility(-0.34, 0.33, -0.38), we decide to reserve it temporarily. In the subsequent model training, we found that the performance of the model decreased after the humidity was removed so it must be kept.

In conclusion, we finally use **hour_of_day, month, weekday, temp, humidity, snow, windspeed, cloudcover, visibility** as the input features to train our model. In addition, we use StandardScaler to Standardize numerical features to accelerate the convergence of the model.

# 3 Model development

## 3.1 logistic regression

Logistic Regression starts by calculating a linear combination of input features using the formula:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

where $x_1, x_2, \ldots, x_n$ are the input features, and $\beta_0, \beta_1, \ldots, \beta_n$ are the model coefficients. The output of the linear equation is passed through a **sigmoid function** to map it to a probability value between 0 and 1. This probability represents the likelihood of the instance belonging to the positive class. The predicted probability is converted into a binary class label based on a threshold (usually 0.5).

Logistic Regression uses the **log-loss (cross-entropy loss)** to optimize the model:

$$\text{Loss} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Here, $y_i$ is the true label, and $\hat{y}_i$ is the predicted probability.

In this problem, in order to get better performance in Logistic regression, we use StandardScaler to Standardize numerical features to accelerate the convergence of the model. Besides, because of the

highly uneven distribution of low bike demand and high bike demand in the data set, I used sklearn's (class_weight='balanced') to solve this problem during training.

**Model Tuning**

In terms of model tuning, we use **GridSearchCV** to find best parameters. We use 5-fold cross-validation to compare different parameters. The parameters we search are **'penalty'**, **'solver'** and **'C'**. Best parameters are: **'C': 10**, **'penalty': 'l1'**, **'solver': 'liblinear'**. And we get the best cross-validation accuracy: **0.8015625**. As it states before, we found that using (class_weight='balanced') can greatly improve the model's performance.

### 3.2 Discriminant analysis: LDA, QDA

To maximize the separability of the two class centers after projection, we aim to maximize the squared distance between the centers:

$$J = \frac{\|w^\top \mu_0 - w^\top \mu_1\|^2}{w^\top \Sigma_0 w + w^\top \Sigma_1 w}.$$

This is the objective of Linear Discriminant Analysis (LDA), which seeks to maximize the ratio of **within-class scatter matrix** to **within-class scatter matrix**, known as the **generalized Rayleigh quotient**.

Quadratic Discriminant Analysis (QDA) unlikes Linear Discriminant Analysis (LDA), it allows each class to have its own covariance matrix. This flexibility makes QDA suitable for datasets where class distributions have different variances and covariances.

Each class $y \in \{0, 1, \ldots, K-1\}$ has a multivariate Gaussian distribution:

$$p(x \mid y = k) = \frac{1}{(2\pi)^{d/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_k)^\top \Sigma_k^{-1}(x - \mu_k)\right),$$

where: $\mu_k$ is the mean vector, $\Sigma_k$ is the covariance matrix, $d$ is the number of features.

In this case, LDA and QDA are used to reduce the dimensionality of the feature space while preserving as much of the class-discriminative information as possible.

The features (e.g., hour_of_day, temperature, month) are projected onto a lower-dimensional space where the separation between the two classes(increase_stock or not) is maximized.

**Model Tuning**

To find the best performance of the LDA and QDA models in this task, Grid Search has been applied to both models. For LDA, we evaluated different solvers, including **svd**, which uses Singular Value Decomposition and is efficient for high-dimensional datasets; **lsqr**, which employs Least Squares QR decomposition and works well for datasets of moderate size; and **eigen**, which leverages Eigenvalue Decomposition and can provide higher precision for certain data distributions. The best performance for LDA was achieved with the solver **svd**. For QDA, we tuned the **reg_param** (regularization parameter), testing values ranging from 0.6 to 1.0, to balance overfitting and improve the model's robustness. The optimal performance for QDA was obtained with **reg_param** = 0.8. By systematically testing these configurations, we identified the optimal parameters for each model, maximizing their classification performance.

### 3.3 Tree-Based Methods

#### 3.3.1 Classification Tree

**Initial Model Analysis** We began by default DecisionTreeClassifier parameters without restrictions, the initial model had a low F1-score(0.54) due to poor recall for class 1(0.55) caused by class imbalance favoring the majority class 0.

**Motivations of the choices made** To improve performance, we applied grid search with cross-validation to tune parameters. We also addressed class imbalance by assigning a higher weight

to class 1 using class_weight='balanced'. While the adjusted model's accuracy slightly dropped from 84% to 83%, this trade-off significantly boosted the F1-score from 0.51 to 0.66, with class 1 recall improving dramatically from 0.54 to 0.88. The adjustments made the model more balanced in handling both classes, enhancing its practical usability.

**Optimization** The optimal parameters included limiting tree depth to reduce overfitting, increasing min_samples_split and min_samples_leaf to control complexity. Five-fold cross-validation gave an average accuracy of 79.92% with a low standard deviation of 3.2%, improved stability across different data splits.

**Mathematical description** Take the first split condition as example: hour_of_day $\leq 0.5$. This split divides the dataset into two regions. For region $R_1$:

$$\hat{\pi}_{1,0} = \frac{\text{Samples in } R_1 \text{ with class } 0}{\text{Total samples in } R_1} = 0.989, \quad \hat{\pi}_{1,1} = \frac{\text{Samples in } R_1 \text{ with class } 1}{\text{Total samples in } R_1} = 0.011.$$

Similarly for region $R_2$: $\hat{\pi}_{2,0} = 0.648, \quad \hat{\pi}_{2,1} = 0.352.$

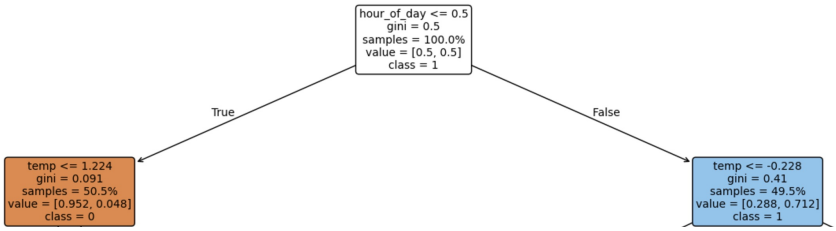



Figure 3: Decision Tree Visualisation

The Gini impurity is calculated using:

$$Gini = 1 - \sum_{m=0}^{1} \hat{\pi}_m^2.$$

$R_1 : Q_1 = 1 - \left(\hat{\pi}_{1,0}^2 + \hat{\pi}_{1,1}^2\right) = 1 - \left(0.989^2 + 0.011^2\right) = 0.021$, $R_2 : Q_2 = 0.456$.

$$\text{Total Weighted Gini Impurity} = \frac{270}{320} \cdot 0.021 + \frac{50}{320} \cdot 0.456 = 0.237.$$

**Why `hour_of_day` was chosen** The split hour_of_day $\leq 0.5$ minimizes the weighted Gini impurity (0.237) among all features and possible split points.

### 3.3.2 Bagging

Bagging is an ensemble learning method that reduces the variance of the model without increasing its bias. Bagging achieves this by training multiple decision trees on different bootstrapped subsets of the data and combining their predictions. To classify a new $x$, bagging uses majority voting among the models:

$$\hat{y} = \text{majority_vote}\left(f_1(x), f_2(x), \ldots, f_B(x)\right).$$

Bagging reduces the variance by averaging across multiple independent models:

$$\text{Var}(\hat{f}(x)) = \frac{\sigma^2}{B},$$

where $\sigma^2$ is the variance of a single model, and $B$ is the number of models in the ensemble.

The Bagging approach achieved a slight improvement in the weighted F1 score of 84. 8% to 85. 6%, which reduces the variance for this classification.

### 3.3.3 Random Forest

Random forests improve bagging by using a random subset of features at each split, reducing tree correlation, and improving variance reduction through prediction averaging. Similarly to the bagging method, the Random Forest method aggregates predictions from all decision trees in the forest. Random Forest reduces model variance by averaging predictions across multiple trees:

$$\text{Var}(\hat{f}(x)) = \frac{\rho\sigma^2}{N} + (1-\rho)\sigma^2,$$

where $\rho$ is the correlation between trees, $\sigma^2$ is the variance of a single tree, and $N$ is the number of trees.

The tuned random forest model achieved a weighted F1-score of 86.57% and significantly boosted recall for class 1 to 93%, while maintaining strong performance for class 0 at 84%.

## 3.4 Boosting

Boosting is a family of algorithms that transform weak learners into strong learners. First, a base learner is trained, and based on its performance, the training sample distribution is adjusted so that misclassified samples receive more attention in the next iteration. Then, a new base learner is trained on the adjusted sample distribution. This process is repeated until the predefined number of learners $T$ is reached, and finally, these learners are combined using weighted aggregation.

The most famous representative of the Boosting family is AdaBoost, which is based on the "additive model," that is, a linear combination of base learners.

The final strong classifier in AdaBoost is a linear combination of weak classifiers:

$$F(x) = \sum_{m=1}^{M} \alpha_m h_m(x)$$

Where $\epsilon_m$ is the error rate of the $m$-th weak classifier, $F(x)$ is the final strong classifier, $M$ is the number of weak classifiers, $\alpha_m$ is the weight of the $m$-th weak classifier $h_m(x)$, and $h_m(x)$ is the $m$-th weak classifier

The weight $\alpha_m$ is computed based on the error rate of the weak classifier:

$$\alpha_m = \frac{1}{2}\ln\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$$

where $\epsilon_m$ is the error rate of the $m$-th weak classifier.

Optimized Model Analysis For our optimized model analysis, we initially selected decision trees as weak learners. We applied grid search with cross-validation to tune key hyperparameters, including max_depth, min_samples_split, and min_samples_leaf. To address class imbalance, we set the class_weight parameter to 'balanced', which assigns higher weights to the minority class (class 1). Next, we followed a similar approach to tune the parameters of the AdaBoost model, specifically focusing on the n_estimators (number of estimators) and learning_rate. These adjustments resulted in a more balanced model overall.

Grid Search Results

The optimal parameters involved constraining the tree depth to prevent overfitting, raising the minimum samples required for splits and leaves to manage model complexity, and limiting the number of estimators and learning rate to minimize overfitting risks. Five-fold cross-validation yielded an average F1 score of 0.84, with a low standard deviation of 2.3%, indicating the model,Äôs consistency across different data partitions.

## 4 Conclusion

In this project we focused on predicting whether the number of bikes in a sharing system should be increased using machine learning methods. After analyzing the dataset, we conducted feature selection and explored various models, including Logistic Regression, Discriminant Analysis, Tree-Based Methods, and Boosting.

Among these methods, **QDA** achieved the best performance1, which shows the **best F1-score** of **category 1 & 0 & overall** and best accuracy.

During the project, it can be found that the key strategies, such as feature engineering, hyperparameter tuning, and class weighting, were critical to improving the model's accuracy and robustness.

Table 1: Performance metrics for various models

| **Models** | **F1-score (0)** | **F1-score (1)** | **F1-score (Overall)** | **Accuracy** |
|---|---|---|---|---|
| Dummy | 0.61 | 0.23 | 0.5373 | 0.48 |
| Logistic Regression | 0.89 | 0.68 | 0.8507 | 0.83 |
| LDA | 0.92 | 0.61 | 0.8649 | 0.87 |
| QDA | **0.93** | **0.72** | **0.8914** | **0.89** |
| Classification Trees | 0.89 | 0.66 | 0.8485 | 0.83 |
| Random Forests | 0.90 | 0.70 | 0.8657 | 0.85 |
| Boosting | 0.93 | 0.59 | 0.8725 | 0.88 |
| Bagging | 0.92 | 0.55 | 0.8560 | 0.87 |

## References


[1] Tibshirani, R. (1996). "Regression shrinkage and selection via the LASSO." *Journal of the Royal Statistical Society: Series B*, 58(1), 267-288.

[2] Fisher, R. A. (1936). "The use of multiple measurements in taxonomic problems." *Annals of Eugenics*, 7(2), 179-188.

[3] Zhang, M.-L., & Zhou, Z.-H. (2014). "A review on multi-label learning algorithms." *IEEE Transactions on Knowledge and Data Engineering*, 26(1), 1-18.

[4] Zhou, Z.-H., & Liu, X.-Y. (2006). "Training cost-sensitive neural networks with methods addressing the class imbalance problem." *IEEE Transactions on Knowledge and Data Engineering*, 18(11), 1495-1508.


## Main code

# data_pre_processing

December 22, 2024

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

[15]:
```
# Load data
file_path = "training_data_fall2024.csv"  # Replace with your file path
data = pd.read_csv(file_path)

# Convert 'increase_stock' to binary: 1 for 'high_bike_demand', 0 otherwise
# data['increase_stock'] = data['increase_stock'].apply(lambda x: 1 if x ==␣
↪'high_bike_demand' else 0)

# Convert 'snow' based on 'snowdepth': 1 if snow depth is not 0, otherwise 0
data['snow'] = data['snowdepth'].apply(lambda x: 1 if x != 0 else 0)

# Convert 'hour_of_day' to binary: 1 if between 8 AM and 8 PM, otherwise 0
data['hour_of_day'] = data['hour_of_day'].apply(lambda x: 1 if 8 <= x < 20 else␣
↪0)

# Convert 'month' into a sine function
data['month'] = np.sin(2 * np.pi * ((data['month']) % 12) / 12)

# Drop columns that are considered unnecessary
data = data.drop(columns=['snowdepth', 'summertime', 'dew', 'day_of_week',␣
↪'precip', 'holiday'])

# Normalize selected columns
standard_cols = ['temp', 'humidity', 'windspeed', 'cloudcover', 'visibility']

# Initialize StandardScaler
scaler = StandardScaler()

# Apply normalization
data[standard_cols] = scaler.fit_transform(data[standard_cols])
```

```
# Save the processed data to a CSV file
data.to_csv('MonthToSin_Standard_NoHoliday/data.csv', index=False)
# Uncomment this line if you want to save with a different name
# data.to_csv('result_data_set_HourToSin.csv', index=False)
```

[16]:
```
train, test = train_test_split(data, test_size=0.2, stratify=data.increase_stock)
print(train)
print(test)
train.to_csv('MonthToSin_Standard_NoHoliday/train.csv', index=False)
test.to_csv('MonthToSin_Standard_NoHoliday/test.csv', index=False)
```

```
      hour_of_day     month  weekday      temp  humidity  snow  windspeed  \
780             0 -1.000000        1  0.441561  1.598145     0  -0.281459
930             1  0.000000        0  0.398373 -0.886459     0  -0.294356
1598            1  1.000000        0 -0.379006 -2.193511     0  -0.939165
456             0 -0.500000        0 -1.469496  0.448380     0  -0.294356
1216            0 -0.866025        1  0.959814  1.309262     0  -1.635559
...           ...       ...      ...       ...       ...   ...        ...
711             1  0.866025        0 -0.432991 -1.089883     0   1.188706
1023            0  0.500000        0 -0.756899  0.876724     0   2.001166
781             0  0.866025        0 -1.782607  1.326040     1  -0.191186
1543            0 -0.866025        1  0.916626  1.269416     0  -0.513591
1418            0  1.000000        1 -1.231964 -1.697534     0   1.162914

      cloudcover  visibility    increase_stock
780     0.680469    0.282338   low_bike_demand
930     0.518581    0.282338  high_bike_demand
1598   -1.219427    0.282338  high_bike_demand
456    -1.964724    0.282338   low_bike_demand
1216    1.053117    0.282338   low_bike_demand
...          ...         ...               ...
711     0.488036    0.282338  high_bike_demand
1023    1.089771   -0.879947   low_bike_demand
781     1.089771   -2.343564   low_bike_demand
1543    0.588834    0.239291   low_bike_demand
1418    0.616325    0.282338   low_bike_demand

[1280 rows x 10 columns]
      hour_of_day     month  weekday      temp  humidity  snow  windspeed  \
1564            1 -0.500000        1 -0.098286  1.732888     0  -0.526487
1451            1 -0.866025        1 -0.497772  1.473889     0  -0.268563
557             0  0.866025        1 -0.044301 -0.086919     0  -1.016542
1160            1 -0.866025        1  1.575239 -0.027150     0  -0.062224
800             0  0.866025        1  1.121768 -0.831933     0   0.208596
...           ...       ...      ...       ...       ...   ...        ...
1458            1  1.000000        1 -0.152271 -1.880511     0   2.014063
178             0  1.000000        1 -1.782607 -1.546015     0   1.562696
```

```
1558            1  0.000000        0 -1.048416  1.149354     0  -0.732826
735             1  1.000000        1 -0.854071 -0.206981     0   0.659963
371             1 -0.866025        1  0.981408 -0.296634     0  -0.062224

      cloudcover  visibility    increase_stock
1564    1.089771   -6.562228  high_bike_demand
1451    1.089771   -0.922994   low_bike_demand
557     0.915665    0.282338   low_bike_demand
1160   -0.464967    0.282338   low_bike_demand
800     1.053117    0.282338   low_bike_demand
...          ...         ...               ...
1458   -1.964724    0.282338   low_bike_demand
178    -1.219427    0.282338   low_bike_demand
1558    1.089771    0.282338   low_bike_demand
735     0.900393    0.282338   low_bike_demand
371     0.787376    0.282338  high_bike_demand

[320 rows x 10 columns]
```

[ ]:

# Logistic Regression

December 22, 2024

[3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold

df_train = pd.read_csv('train.csv')
df_test = pd.read_csv('test.csv')

train_X = df_train.drop(columns=["increase_stock"])
train_Y = df_train["increase_stock"]

test_X = df_test.drop(columns=["increase_stock"])
test_Y = df_test["increase_stock"]
```

## 0.1 No Weight Balance

[4]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix, f1_score
from sklearn.model_selection import KFold, cross_val_score

# Create a Logistic Regression model
# model = LogisticRegression(max_iter=500)

# Logistic Regression model with L1 regularization
model = LogisticRegression(
    max_iter=500    # Increase the number of iterations to ensure convergence
)

# Train the model
model.fit(train_X, train_Y)

# Predict the test set
pred_Y = model.predict(test_X)
```

```
# Initialize 5-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Perform cross-validation and calculate accuracy scores
scores = cross_val_score(model, train_X, train_Y, cv=kf, scoring='accuracy')
print(f'Logistic Regression 5-fold Accuracy: {scores.mean():.4f} ± {scores.std():
 ↪.4f}')

# Model evaluation
accuracy = accuracy_score(test_Y, pred_Y)
print("Model accuracy:", accuracy)

print("\nClassification Report:")
print(classification_report(test_Y, pred_Y))
print("F1 Score:", f1_score(test_Y, pred_Y))

print("\nConfusion Matrix:")
print(confusion_matrix(test_Y, pred_Y))
```

```
Logistic Regression 5-fold Accuracy: 0.8648 ± 0.0161
Model accuracy: 0.871875

Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.94      0.92       262
           1       0.68      0.55      0.61        58

    accuracy                           0.87       320
   macro avg       0.79      0.75      0.77       320
weighted avg       0.86      0.87      0.87       320

F1 Score: 0.6095238095238096

Confusion Matrix:
[[247  15]
 [ 26  32]]
```

## 0.2 Weight Balance

[5]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix, f1_score
from sklearn.model_selection import KFold, cross_val_score

# Create a Logistic Regression model
```

```
# model = LogisticRegression(max_iter=500)

# Logistic Regression model with L1 regularization
model = LogisticRegression(
    max_iter=500,    # Increase the number of iterations to ensure convergence
    class_weight = 'balanced'
)

# Train the model
model.fit(train_X, train_Y)

# Predict the test set
pred_Y = model.predict(test_X)

# Initialize 5-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Perform cross-validation and calculate accuracy scores
scores = cross_val_score(model, train_X, train_Y, cv=kf, scoring='accuracy')
print(f'Logistic Regression 5-fold Accuracy: {scores.mean():.4f} ± {scores.std():
 ↪.4f}')

# Model evaluation
accuracy = accuracy_score(test_Y, pred_Y)
print("Model accuracy:", accuracy)

print("\nClassification Report:")
print(classification_report(test_Y, pred_Y))
print("F1 Score:", f1_score(test_Y, pred_Y))

print("\nConfusion Matrix:")
print(confusion_matrix(test_Y, pred_Y))
```

```
Logistic Regression 5-fold Accuracy: 0.7992 ± 0.0277
Model accuracy: 0.834375

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.80      0.89       262
           1       0.52      0.98      0.68        58

    accuracy                           0.83       320
   macro avg       0.76      0.89      0.79       320
weighted avg       0.91      0.83      0.85       320

F1 Score: 0.6826347305389222
```

```
Confusion Matrix:
[[210  52]
 [  1  57]]
```

### 0.3 GridSearchCV

[9]:
```python
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),  #
    ('logreg', LogisticRegression(max_iter=500, class_weight='balanced'))  #
])

#
param_grid = {
    'logreg__C': [0.01, 0.1, 1, 10, 100],  #
    'logreg__penalty': ['l1', 'l2'],  #
    'logreg__solver': ['liblinear', 'saga']  #
}

#
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,  # 5
    scoring='accuracy',  #
    verbose=1,  #
    n_jobs=-1  #  CPU
)

#
grid_search.fit(train_X, train_Y)

#
print("Best parameters:", grid_search.best_params_)
print("Best cross-validation accuracy:", grid_search.best_score_)

#
best_model = grid_search.best_estimator_
pred_Y = best_model.predict(test_X)

#
from sklearn.metrics import accuracy_score, classification_report
```

```
print("Test Accuracy:", accuracy_score(test_Y, pred_Y))
print("\nClassification Report:\n", classification_report(test_Y, pred_Y))
```

```
Fitting 5 folds for each of 20 candidates, totalling 100 fits
Best parameters: {'logreg__C': 10, 'logreg__penalty': 'l1', 'logreg__solver':
'liblinear'}
Best cross-validation accuracy: 0.8015625
Test Accuracy: 0.83125

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.80      0.89       262
           1       0.52      0.98      0.68        58

    accuracy                           0.83       320
   macro avg       0.76      0.89      0.78       320
weighted avg       0.91      0.83      0.85       320
```

# LDA_QDA

December 22, 2024

[356]:
```
import pandas as pd
import os

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,␣
 ↪QuadraticDiscriminantAnalysis
from sklearn.dummy import DummyClassifier
from sklearn.metrics import f1_score, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.utils import resample
from sklearn.model_selection import GridSearchCV
```

[357]:
```
def separate_features_target(data):
    scaler = StandardScaler()
    X = data.drop('increase_stock', axis=1)
    X = X.drop('holiday', axis=1)
    y = data['increase_stock']
    X = scaler.fit_transform(X)
    return X, y
```

[358]:
```
def load_data(path):
    train_path = os.path.join(path, 'train.csv')
    test_path = os.path.join(path, 'test.csv')
    train_data = pd.read_csv(train_path)
    test_data = pd.read_csv(test_path)
    X_train, y_train = separate_features_target(train_data)
    X_test, y_test = separate_features_target(test_data)
    return X_train, y_train, X_test, y_test
```

[359]:
```
def train_model(X_train, y_train, model, param_grid):
    grid_search = GridSearchCV(estimator=model,
                               param_grid=param_grid,
                               cv=5,
                               scoring='f1_weighted',
                               n_jobs=-1)
    grid_search.fit(X_train, y_train)
```

```
    print(f'The best parameters of model {model} are: {grid_search.
↪best_params_}')
    print(f'The best score of model {model} is: {grid_search.best_score_}')
    print(f'The cross_entropy of model {model} is: {grid_search.
↪cv_results_["mean_test_score"]}')
    best_model = grid_search.best_estimator_
    return best_model
```

[360]:
```
def Dummy_classifier(X_train, y_train):
    dummy = DummyClassifier(strategy='uniform', random_state=42)
    dummy.fit(X_train, y_train)
    return dummy
```

[361]:
```
def predict_model(X_test, y_test, model):
    y_pred = model.predict(X_test)
    f1 = f1_score(y_test, y_pred, average='weighted')
    print(f'model {model} performance:')
    print(classification_report(y_test, y_pred))
    print(f'f1 score: {f1}')
    return y_pred
```

[362]:
```
def qda_predict_y(model, data_path,output_path):
    X = pd.read_csv(data_path, index_col=False)
    scaler = StandardScaler()
    X = scaler.fit_transform(X)
    X = pd.DataFrame(X)
    y_pred = model.predict(X)
    y_pred = pd.DataFrame(y_pred)
    y_pred = y_pred.T
    y_pred.to_csv(output_path, index=False, header=False)
```

[363]:
```
def main():
    param_grid_lda = {
        'solver': ['svd', 'lsqr', 'eigen']
    }

    param_grid_qda = {
        'reg_param': [0.5, 0.6, 0.7, 0.8, 0.9]
    }
    path = '/home/younger/SML/SML-PRPJECT/MonthToSin_Standard'
    X_train, y_train, X_test, y_test = load_data(path)
    lda = LinearDiscriminantAnalysis()
    qda = QuadraticDiscriminantAnalysis()
    dummy = Dummy_classifier(X_train, y_train)
    models = [lda, qda, dummy]
```

```
    param_grids = [param_grid_lda, param_grid_qda, None]

    for model, param_grid in zip(models, param_grids):
        if param_grid is not None:
            best_model = train_model(X_train, y_train, model, param_grid)
        else:
            best_model = model
        predict_model(X_test, y_test, best_model)
        print('-----------------------------------------------------')


    best_model = train_model(X_train, y_train, qda, param_grid_qda)

    data_path = './data.csv'
    #data_path = '/home/younger/SML/SML-PRPJECT/MonthToSin_Standard/test.csv'
    output_path = './output.csv'
    qda_predict_y(best_model, data_path, output_path)
```

[364]:
```
if __name__ == '__main__':
    main()
```

```
The best parameters of model LinearDiscriminantAnalysis() are: {'solver':
'lsqr'}
The best score of model LinearDiscriminantAnalysis() is: 0.8583319615948295
The cross_entropy of model LinearDiscriminantAnalysis() is: [0.85737607
0.85833196 0.85833196]
model LinearDiscriminantAnalysis(solver='lsqr') performance:
              precision    recall  f1-score   support

           0       0.91      0.94      0.92       262
           1       0.67      0.59      0.62        58

    accuracy                           0.87       320
   macro avg       0.79      0.76      0.77       320
weighted avg       0.87      0.87      0.87       320

f1 score: 0.8686054095613261
-----------------------------------------------------
The best parameters of model QuadraticDiscriminantAnalysis() are: {'reg_param':
0.7}
The best score of model QuadraticDiscriminantAnalysis() is: 0.8610687681455829
The cross_entropy of model QuadraticDiscriminantAnalysis() is: [0.85388115
0.86064003 0.86106877 0.85802106 0.85231181]
model QuadraticDiscriminantAnalysis(reg_param=0.7) performance:
              precision    recall  f1-score   support
```

```
           0       0.95      0.91      0.93       262
           1       0.66      0.79      0.72        58

    accuracy                           0.89       320
   macro avg       0.80      0.85      0.82       320
weighted avg       0.90      0.89      0.89       320

f1 score: 0.891455078125
--------------------------------------------------------
model DummyClassifier(random_state=42, strategy='uniform') performance:
              precision    recall  f1-score   support

           0       0.80      0.49      0.61       262
           1       0.16      0.43      0.23        58

    accuracy                           0.48       320
   macro avg       0.48      0.46      0.42       320
weighted avg       0.68      0.48      0.54       320

f1 score: 0.5372709470427384
--------------------------------------------------------
The best parameters of model QuadraticDiscriminantAnalysis() are: {'reg_param':
0.7}
The best score of model QuadraticDiscriminantAnalysis() is: 0.8610687681455829
The cross_entropy of model QuadraticDiscriminantAnalysis() is: [0.85388115
0.86064003 0.86106877 0.85802106 0.85231181]

/home/younger/miniconda/envs/d2l_copy/lib/python3.9/site-
packages/sklearn/discriminant_analysis.py:947: UserWarning: Variables are
collinear
  warnings.warn("Variables are collinear")
/home/younger/miniconda/envs/d2l_copy/lib/python3.9/site-
packages/sklearn/discriminant_analysis.py:947: UserWarning: Variables are
collinear
  warnings.warn("Variables are collinear")
/home/younger/miniconda/envs/d2l_copy/lib/python3.9/site-
packages/sklearn/discriminant_analysis.py:947: UserWarning: Variables are
collinear
  warnings.warn("Variables are collinear")
/home/younger/miniconda/envs/d2l_copy/lib/python3.9/site-
packages/sklearn/discriminant_analysis.py:947: UserWarning: Variables are
collinear
  warnings.warn("Variables are collinear")
/home/younger/miniconda/envs/d2l_copy/lib/python3.9/site-
packages/sklearn/discriminant_analysis.py:947: UserWarning: Variables are
collinear
  warnings.warn("Variables are collinear")
```

# Tree_Based_Methods

December 22, 2024

[1]:
```
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

from sklearn import tree
from sklearn.model_selection import GridSearchCV, cross_val_score,␣
↪RandomizedSearchCV
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, precision_score,␣
↪recall_score, classification_report

import graphviz
```

[2]:
```
random_state_setting = 42

df_train = pd.read_csv('data/train_new.csv')
df_test = pd.read_csv('data/test_new.csv')

train_X = df_train.drop(columns=["increase_stock"])
train_Y = df_train["increase_stock"]

test_X = df_test.drop(columns=["increase_stock"])
test_Y = df_test["increase_stock"]
```

## 0.1 1. Classification Tree

[3]:
```
# Basic Decision Tree Classifier
model_basic = tree.DecisionTreeClassifier(random_state=random_state_setting)
model_basic.fit(train_X, train_Y)
pred_Y = model_basic.predict(test_X)

print("Decision Tree Accuracy:", accuracy_score(test_Y, pred_Y))
print("Overall F1-Score (Weighted):", f1_score(test_Y, pred_Y))
print("Classification Report:")
print(classification_report(test_Y, pred_Y))
```

```
Decision Tree Accuracy: 0.83125
```

```
Overall F1-Score (Weighted): 0.5423728813559322
Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.89      0.90       262
           1       0.53      0.55      0.54        58

    accuracy                           0.83       320
   macro avg       0.72      0.72      0.72       320
weighted avg       0.83      0.83      0.83       320
```

From the result, we see that the model performs very well for the majority class 0, with a precision, recall, and F1-score of 0.9. This indicates the model is highly confident and accurate when predicting 0.

However, the model performs worse for the minority class 1, the F1-score is 0.51.

This may due to the imbalance of the input data. In the training data "increase_stock", we have more 0s (1312 counts) than 1s (288 counts).

The overall Accuracy is 84%, it is good butlikely skewed toward the majority class 0.

[4]:
```python
# Model tuning
# Define parameter grid
param_grid = {
    'max_depth': range(2, 6),
    'min_samples_split': (2, 20),
    'min_samples_leaf': (2, 20),
    'max_features': [None, 'sqrt', 'log2'],
}

# Initialize GridSearchCV
grid_search = GridSearchCV(
    estimator=model_basic,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

# Fit the grid search on the training data
grid_search.fit(train_X, train_Y)

# Retrieve the best parameters and corresponding score
best_params = grid_search.best_params_
best_score = grid_search.best_score_
```

```
best_max_depth = best_params['max_depth']
best_max_features = best_params['max_features']
best_min_samples_leaf = best_params['min_samples_leaf']
best_min_samples_split = best_params['min_samples_split']

print("Best Parameters:", best_params)
print("Best Cross-Validation Score:", best_score)
```

```
Fitting 5 folds for each of 48 candidates, totalling 240 fits
Best Parameters: {'max_depth': 5, 'max_features': None, 'min_samples_leaf': 20,
'min_samples_split': 2}
Best Cross-Validation Score: 0.8609375
```

[5]:
```
model_tuned = tree.DecisionTreeClassifier(
    random_state=random_state_setting,
    class_weight='balanced', # Handel class imbalance with 'weighted'
    max_depth=best_max_depth, # To avoid severe overfit, we add some␣
 ↪constraints to the tree
    min_samples_leaf=best_min_samples_leaf,
    min_samples_split=best_min_samples_split,
    )
model_tuned.fit(train_X, train_Y)
pred_Y = model_tuned.predict(test_X)

print("Decision Tree Accuracy (Tuned):", accuracy_score(test_Y, pred_Y))
print("Overall F1-Score (Weighted):", f1_score(test_Y, pred_Y, average =␣
 ↪'weighted'))
print("Classification Report (Tuned):")
print(classification_report(test_Y, pred_Y))
```

```
Decision Tree Accuracy (Tuned): 0.834375
Overall F1-Score (Weighted): 0.8485525440638512
Classification Report (Tuned):
              precision    recall  f1-score   support

           0       0.97      0.82      0.89       262
           1       0.53      0.88      0.66        58

    accuracy                           0.83       320
   macro avg       0.75      0.85      0.77       320
weighted avg       0.89      0.83      0.85       320
```

After tuning, the Accuracy decrease from 84% to 80%, but the F1-score improved from 0.51 to 0.58. the recall of class 1 improved from 0.54 to 0.86.

Since we increases the weight for the minority class, we improved recall for class 1 at the price of precision and overall accuracy.

This makes the model more sensitive to the minority class but less confident in predictions for the

majority class.This balanced both classes, lowering accuracy but improving fairness.

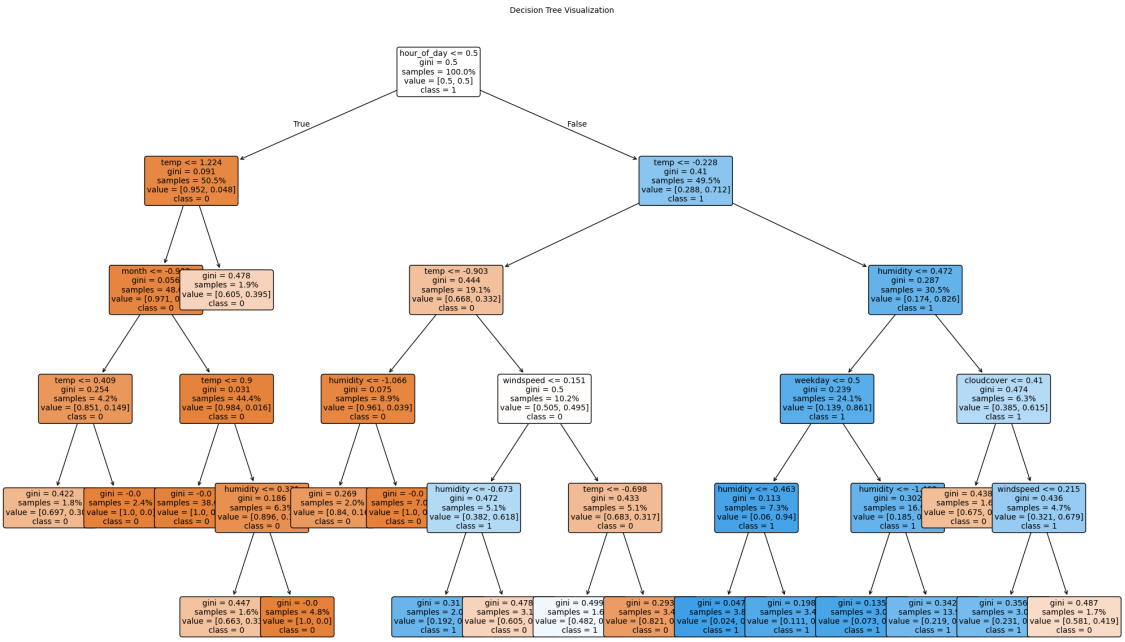
[6]:
```
def cross_validation(X, y, cv, model=None):
  # Evaluate the Tuned Model using Cross-Validation
  # Perform 4-fold cross-validation
  cv_scores = cross_val_score(
      estimator=model,  # Tuned model
      X=X,  # Features
      y=y,  # Target
      cv=cv,  # Number of folds
      scoring='accuracy',  # Evaluation metric
      n_jobs=-1  # Use all processors for speed
  )

  # Print the results
  print("Cross-Validation Scores:", cv_scores)
  print("Mean CV Accuracy:", cv_scores.mean())
  print("Standard Deviation of CV Scores:", cv_scores.std())
```

[7]:
```
cross_validation(train_X, train_Y, cv=5, model=model_tuned)
```

```
Cross-Validation Scores: [0.8125     0.79296875 0.79296875 0.81640625 0.8203125
]
Mean CV Accuracy: 0.80703125
Standard Deviation of CV Scores: 0.011744762795603833
```

[8]:
```
# Use graphviz to visualize the tree
plt.figure(figsize=(25, 15))
tree.plot_tree(model_tuned, feature_names=train_X.columns, class_names=["0",␣
 ↪"1"], fontsize=10, filled=True, rounded=True, proportion=True)
plt.title("Decision Tree Visualization", fontsize=10)
plt.show()
```




Take the first node on the right side for example, we can interpret the information displayed.

temp <= 12.5 the data splited based on the feature temp

gini = 0.405 this node is moderately mixed but leans toward one class.

samples = 49.4% this node contains 49.4% of all the training data points.

value = [0.283, 0.717] 28.3% of the samples belong to Class 0 (no increase),71.7% of the samples belong to Class 1 (increase)

class = 1 The majority class in this node is Class 1 (increase stock). This is the predicted class for samples that end at this node.

[9]:
```
pd.crosstab(test_Y, pred_Y, rownames=['Actual'], colnames=['Predicted'],␣
↪margins=True)
```

[9]:

| Predicted | 0 | 1 | All |
|---|---|---|---|
| Actual | | | |
| 0 | 216 | 46 | 262 |
| 1 | 7 | 51 | 58 |
| All | 223 | 97 | 320 |

The model performs well for class 0, and good for class 1. It predicts more False Positives. The trade-off is between precision and recall. The model favors recall for class 1 at the expense of predicting 1 accurately. If missing class 1 predictions is costly, then it is fair to choose towards recall.

Therefore, while the accuracy of 81.17% seems reasonable, it is influenced by the large number of

majority class 0.

### 0.2 2. Bagging Approach

[10]:
```
# Use bagging classifier
estimator_basic = tree.
 ↪DecisionTreeClassifier(random_state=random_state_setting,␣
 ↪class_weight='balanced')
model_bagging = BaggingClassifier(estimator_basic,␣
 ↪random_state=random_state_setting)
model_bagging.fit(train_X, train_Y)
pred_Y_bagging = model_bagging.predict(test_X)

print("Bagging Accuracy:", accuracy_score(test_Y, pred_Y_bagging))
print("Overall F1-Score (Weighted):", f1_score(test_Y, pred_Y_bagging))
print("Classification Report:")
print(classification_report(test_Y, pred_Y_bagging))
```

```
Bagging Accuracy: 0.871875
Overall F1-Score (Weighted): 0.594059405940594
Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.95      0.92       262
           1       0.70      0.52      0.59        58

    accuracy                           0.87       320
   macro avg       0.80      0.73      0.76       320
weighted avg       0.86      0.87      0.86       320
```

[11]:
```
# Tuning with grid-search
base_estimator = tree.DecisionTreeClassifier(random_state=random_state_setting,␣
 ↪class_weight='balanced')

# Define the BaggingClassifier
bagging_model = BaggingClassifier(estimator=base_estimator,␣
 ↪random_state=random_state_setting)

param_grid = {
    'n_estimators': [10, 50, 100],
    'max_samples': [0.5, 0.8, 1.0],
    'max_features': [0.5, 0.8, 1.0]
}

grid_search = GridSearchCV(
    estimator=bagging_model,
```

```
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

# Fit the grid search
grid_search.fit(train_X, train_Y)

# Retrieve the best parameters and score
best_params = grid_search.best_params_
best_score = grid_search.best_score_

best_max_features=best_params['max_features']
best_max_samples=best_params['max_samples']
best_n_estimators=best_params['n_estimators']

print("Best Parameters:", best_params)
print(f"Best Cross-Validation Accuracy: {best_score:.2f}")
```

```
Best Parameters: {'max_features': 0.8, 'max_samples': 1.0, 'n_estimators': 50}
Best Cross-Validation Accuracy: 0.86
```

[12]:
```
model_bagging = BaggingClassifier(estimator=base_estimator,␣
 ↪random_state=random_state_setting,
                                  max_features=best_max_features,␣
 ↪max_samples=best_max_samples, n_estimators=best_n_estimators
)
model_bagging.fit(train_X, train_Y)
pred_Y_bagging_tune = model_bagging.predict(test_X)

print("Bagging Accuracy (Tuned):", accuracy_score(test_Y, pred_Y_bagging_tune))
print("Overall F1-Score (Weighted):", f1_score(test_Y, pred_Y_bagging_tune))
print("Classification Report (Tuned):")
print(classification_report(test_Y, pred_Y_bagging_tune))
```

```
Bagging Accuracy (Tuned): 0.86875
Overall F1-Score (Weighted): 0.5531914893617021
Classification Report (Tuned):
              precision    recall  f1-score   support

           0       0.89      0.96      0.92       262
           1       0.72      0.45      0.55        58

    accuracy                           0.87       320
   macro avg       0.80      0.71      0.74       320
weighted avg       0.86      0.87      0.86       320
```

[13]:
```
cross_validation(train_X, train_Y, cv=4, model=model_bagging)
```

```
Cross-Validation Scores: [0.8625   0.865625 0.84375  0.85     ]
Mean CV Accuracy: 0.85546875
Standard Deviation of CV Scores: 0.008941814954890315
```

[14]:
```
pd.crosstab(test_Y, pred_Y_bagging_tune, rownames=['Actual'],␣
↪colnames=['Predicted'], margins=True)
```

[14]:

| Predicted | 0 | 1 | All |
|---|---|---|---|
| Actual | | | |
| 0 | 252 | 10 | 262 |
| 1 | 32 | 26 | 58 |
| All | 284 | 36 | 320 |

The bagging approach, even after tuning, provides only marginal improvements in F1-Score for Class 1 and achieves slightly better recall for Class 1 (from 0.48 to 0.56).

Class 1 has a much smaller sample size compared to Class 0. This makes it difficult for the model to accurately classify Class 1. Bagging reduces overfitting but doesn't solve class imbalance. Since it uses bootstrap sampling, it might even exacerbate the imbalance by under-sampling the minority class within each bootstrap sample.

### 0.3 3. Random Forests

[15]:
```
# Use random forests classifier
rf_model = RandomForestClassifier(random_state=random_state_setting,␣
↪class_weight='balanced')

# Train the model
rf_model.fit(train_X, train_Y)

# Predict on the test set
pred_Y_rf = rf_model.predict(test_X)

# Evaluate the model
print("Random Forest Accuracy:", accuracy_score(test_Y, pred_Y_rf))
print("Overall F1-Score (Weighted):", f1_score(test_Y, pred_Y_rf, ))
print("Classification Report:")
print(classification_report(test_Y, pred_Y_rf))
```

```
Random Forest Accuracy: 0.88125
Overall F1-Score (Weighted): 0.6274509803921569
Classification Report:
              precision    recall  f1-score   support

           0       0.91      0.95      0.93       262
           1       0.73      0.55      0.63        58
```

```
    accuracy                           0.88       320
   macro avg       0.82      0.75      0.78       320
weighted avg       0.87      0.88      0.87       320
```

[16]:
```
# Tuning with grid-search
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': range(2, 6),
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None],
}

# Initialize GridSearchCV
grid_search_rf = GridSearchCV(
    estimator=rf_model,
    param_grid=param_grid,
    cv=5,
    scoring='f1_weighted',
    n_jobs=-1,
    verbose=1
)

# Fit the grid search
grid_search_rf.fit(train_X, train_Y)

# Retrieve the best parameters and model
best_rf_params = grid_search_rf.best_params_
best_rf_model = grid_search_rf.best_estimator_

print("Best Parameters:", best_rf_params)
```

```
Fitting 5 folds for each of 324 candidates, totalling 1620 fits
Best Parameters: {'max_depth': 5, 'max_features': None, 'min_samples_leaf': 2,
'min_samples_split': 2, 'n_estimators': 50}
```

[17]:
```
pred_Y_rf_tune = best_rf_model.predict(test_X)
print("Random Forest Accuracy (Tuned):", accuracy_score(test_Y, pred_Y_rf_tune))
print("Overall F1-Score (Weighted):", f1_score(test_Y, pred_Y_rf_tune, average␣
 ↪= 'weighted'))
print("Classification Report (Tuned):")
print(classification_report(test_Y, pred_Y_rf_tune))
```

```
Random Forest Accuracy (Tuned): 0.85625
Overall F1-Score (Weighted): 0.8683655336433114
Classification Report (Tuned):
              precision    recall  f1-score   support
```

```
              0       0.98      0.84      0.91       262
              1       0.56      0.93      0.70        58

    accuracy                            0.86       320
   macro avg       0.77      0.89      0.80       320
weighted avg       0.91      0.86      0.87       320
```

[18]:
```
def cross_validation(X, y, cv, model=None):
  # Evaluate the Tuned Model using Cross-Validation
  # Perform 5-fold cross-validation
  cv_scores = cross_val_score(
      estimator=model,  # Tuned model
      X=X,  # Features
      y=y,  # Target
      cv=cv,  # Number of folds
      scoring='accuracy',  # Evaluation metric
      n_jobs=-1  # Use all processors for speed
  )

  # Print the results
  print("Cross-Validation Scores:", cv_scores)
  print("Mean CV Accuracy:", cv_scores.mean())
  print("Standard Deviation of CV Scores:", cv_scores.std())
```

[19]:
```
cross_validation(train_X, train_Y, cv=5, model=best_rf_model)
```

```
Cross-Validation Scores: [0.83984375 0.828125   0.828125   0.8203125
0.83203125]
Mean CV Accuracy: 0.8296875
Standard Deviation of CV Scores: 0.006346905003621844
```

The overall accuracy drops slightly (from 0.84 to 0.82) because the model sacrifices some precision and recall for the majority Class 0 to improve recall for the minority class.

For Class 0, performance for the majority class is strong, though slightly reduced to accommodate improvements for Class 1.

For Class 1, there are significant improvement in recall, meaning the model now correctly identifies 84% of the actual 1 instances (compared to 54% before tuning). This is a obvious gain in the ability to detect the minority class. Improvement in F1-Score also shows a better balance between precision and recall for the minority class.

After tuning, the model focuses more on detecting the minority Class 1, as indicated by the improvement in recall from 0.54 to 0.84. This holds for imbalanced datasets where the minority class is important.

[20]:
```
pd.crosstab(test_Y, pred_Y_rf_tune, rownames=['Actual'],␣
↪colnames=['Predicted'], margins=True)
```

[20]:
```
Predicted    0   1  All
Actual
0          220  42  262
1            4  54   58
All        224  96  320
```

[21]:
```
# Refer evaluation
#
def evaluate(y_true, y_pred):
    return {
        'accuracy_score': round(accuracy_score(y_true, y_pred), 4),
        'precision_score': round(precision_score(y_true, y_pred,␣
 ↪zero_division=0), 4),
        'recall_score': round(recall_score(y_true, y_pred, zero_division=0), 4),
        'f1_score': round(f1_score(y_true, y_pred, zero_division=0), 4)
    }
```

[22]:
```
# evaluate all tree-models
ct_metrics = evaluate(test_Y, pred_Y)
bg_metrics = evaluate(test_Y, pred_Y_bagging_tune)
rf_metrics = evaluate(test_Y, pred_Y_rf_tune)

print("Classification Tree metrics:", ct_metrics)
print("Bagging metrics:", bg_metrics)
print("Random Forestmetrics:", rf_metrics)
```

```
Classification Tree metrics: {'accuracy_score': 0.8344, 'precision_score':
0.5258, 'recall_score': 0.8793, 'f1_score': 0.6581}
Bagging metrics: {'accuracy_score': 0.8688, 'precision_score': 0.7222,
'recall_score': 0.4483, 'f1_score': 0.5532}
Random Forestmetrics: {'accuracy_score': 0.8562, 'precision_score': 0.5625,
'recall_score': 0.931, 'f1_score': 0.7013}
```

### 0.4 4. Prediction

[24]:
```
# Load test data
df_test_data = pd.read_csv('data/test_data.csv')

# Prediction on test data
rf_final_prediction = best_rf_model.predict(df_test_data)

# Save predictions as a single row in csv file
with open('rf_final_prediction.csv', 'w') as f:
    f.write(','.join(map(str, rf_final_prediction)))
```

# boosting

December 9, 2024

[1]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler, MaxAbsScaler
from sklearn.model_selection import train_test_split, \
    GridSearchCV,cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import f1_score, recall_score, precision_score,␣
↪classification_report
import os
os.chdir('D:/uppsala/2.sml/projcet/data')
```

[4]:
```
train = pd.read_csv("train_2.csv")
test = pd.read_csv("test_2.csv")
```

[3]:
```
X_train, X_test, y_train, y_test =\
      train.iloc[:, :-1], test.iloc[:, :-1], train['increase_stock'],␣
↪test['increase_stock']
```

[20]:
```
X_train.head(3)
```

[20]:

| | hour_of_day | month | weekday | temp | humidity | snow | windspeed | cloudcover | visibility |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0.866025 | 1 | -0.562554 | 0.672776 | 0 | -0.320148 | 1.053117 | 0.282338 |
| 1 | 0 | 0.866025 | 0 | -2.549190 | -1.026444 | 0 | -0.036432 | -1.964724 | 0.282338 |
| 2 | 1 | -0.866025 | 1 | 1.272925 | -1.028017 | 0 | -0.216978 | -0.510784 | 0.282338 |

[7]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import f1_score, classification_report

class DecisionTreeModel:
    def __init__(self, param_grid, scoring='f1_weighted', cv=5, n_jobs=-1):
        self.param_grid = param_grid
```

```
        self.scoring = scoring
        self.cv = cv
        self.n_jobs = n_jobs
        self.grid_search = None
        self.best_estimator_ = None

    def fit(self, X_train, y_train):
        base_estimator = DecisionTreeClassifier(class_weight='balanced',␣
↪random_state=666)
        self.grid_search = GridSearchCV(estimator=base_estimator,
                                        param_grid=self.param_grid,
                                        cv=self.cv,
                                        scoring=self.scoring,
                                        n_jobs=self.n_jobs)
        self.grid_search.fit(X_train, y_train)
        self.best_estimator_ = self.grid_search.best_estimator_
        print(f'Best parameters for DecisionTreeClassifier: {self.grid_search.
↪best_params_}')

    def predict(self, X_test):
        if self.best_estimator_ is None:
            raise Exception("Model has not been fitted yet.")
        return self.best_estimator_.predict(X_test)

    def evaluate(self, X_test, y_test):
        y_pred = self.predict(X_test)
        f1 = f1_score(y_test, y_pred, average='weighted')
        report = classification_report(y_test, y_pred)
        print(f'F1 Score: {f1:.2f}')
        print('Classification Report:\n', report)
        return f1, report

    def importances(self):
        if self.best_estimator_ is None:
            raise Exception("Model has not been fitted yet.")

        feature_importances = self.best_estimator_.feature_importances_
        feature_names = X_train.columns
        importance_df = pd.DataFrame({'Feature': feature_names, 'Importance':␣
↪feature_importances})
        importance_df['Importance'] = importance_df['Importance'].round(4)
        importance_df = importance_df.sort_values('Importance', ascending=False)
        importance_df.index = range(1, len(importance_df) + 1)
        return importance_df

# Example usage
if __name__ == "__main__":
```

```
    # Define parameter grid
    param_grid = {
        'max_depth': [None, 10, 20, 30],
        'min_samples_split': [20, 50, 100],
        'min_samples_leaf': [2, 5, 10]
    }

    # Create model instance and train
    model = DecisionTreeModel(param_grid)
    model.fit(X_train, y_train)

    # Evaluate model
    model.evaluate(X_test, y_test)

    # Output feature importances
    print("Feature importances ranking:", model.importances())
```

```
Best parameters for DecisionTreeClassifier: {'max_depth': None,
'min_samples_leaf': 2, 'min_samples_split': 20}
F1 Score: 0.84
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.86      0.89       262
           1       0.52      0.67      0.59        58

    accuracy                           0.83       320
   macro avg       0.72      0.77      0.74       320
weighted avg       0.85      0.83      0.84       320

Feature importances ranking:        Feature  Importance
1  hour_of_day      0.5102
2         temp      0.2521
3     humidity      0.0991
4    windspeed      0.0569
5   cloudcover      0.0564
6      weekday      0.0114
7        month      0.0104
8   visibility      0.0034
9         snow      0.0000
```

[13]:
```
class AdaBoostModel:
    def __init__(self, base_estimator, param_grid, scoring='f1_weighted', cv=5,␣
 ↪n_jobs=-1):
        self.base_estimator = base_estimator
        self.param_grid = param_grid
```

```
        self.scoring = scoring
        self.cv = cv
        self.n_jobs = n_jobs
        self.grid_search = None
        self.best_estimator_ = None

    def fit(self, X_train, y_train):
        # Create an AdaBoostClassifier with the given base estimator
        boosting_model = AdaBoostClassifier(estimator=self.base_estimator,␣
↪random_state=666)

        # Perform GridSearchCV to find the best hyperparameters
        self.grid_search = GridSearchCV(estimator=boosting_model,
                                        param_grid=self.param_grid,
                                        cv=self.cv,
                                        scoring=self.scoring,
                                        n_jobs=self.n_jobs)
        self.grid_search.fit(X_train, y_train)

        # Store the best estimator from the GridSearch
        self.best_estimator_ = self.grid_search.best_estimator_
        print(f'Best parameters for AdaBoostClassifier: {self.grid_search.
↪best_params_}')

    def predict(self, X_test):
        # Ensure that the model has been fitted before making predictions
        if self.best_estimator_ is None:
            raise Exception("Model has not been fitted yet.")
        return self.best_estimator_.predict(X_test)

    def evaluate(self, X_test, y_test):
        # Evaluate the fitted model on test data
        y_pred = self.predict(X_test)

        # Calculate F1 score and generate a classification report
        f1 = f1_score(y_test, y_pred, average='weighted')
        report = classification_report(y_test, y_pred)
        print(f'F1 Score: {f1:.2f}')
        print('Classification Report:\n', report)
        return f1, report

# Example usage
if __name__ == "__main__":

    # Define the parameter grid for DecisionTree
    param_grid_dt = {
        'max_depth': [None, 10, 50, 100],
```

```
        'min_samples_split': [10, 20, 50, 100],
        'min_samples_leaf': [2, 5, 10]
    }

    # Create an instance of DecisionTreeModel and train it
    dt_model = DecisionTreeModel(param_grid_dt)
    dt_model.fit(X_train, y_train)

    # Define the parameter grid for AdaBoost
    param_grid_ab = {
        'n_estimators': [50, 100, 200, 500],
        'learning_rate': [0.01, 0.1, 1, 10]
    }

    # Create an instance of AdaBoostModel and train it
    ab_model = AdaBoostModel(base_estimator=dt_model.best_estimator_,␣
↪param_grid=param_grid_ab)
    ab_model.fit(X_train, y_train)

    # Evaluate the AdaBoost model
    ab_model.evaluate(X_test, y_test)
```

```
Best parameters for DecisionTreeClassifier: {'max_depth': None,
'min_samples_leaf': 2, 'min_samples_split': 20}

F1 Score: 0.87
Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.96      0.93       262
           1       0.72      0.50      0.59        58

    accuracy                           0.88       320
   macro avg       0.81      0.73      0.76       320
weighted avg       0.87      0.88      0.87       320
```